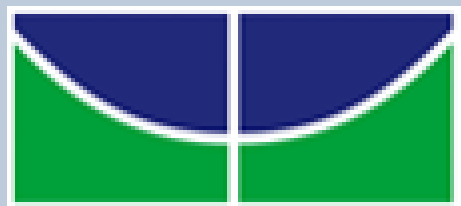


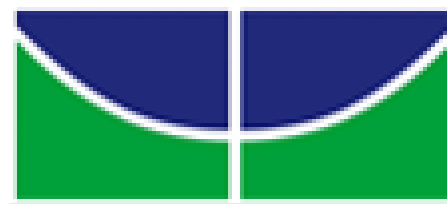
PSPD – Contexto geral do curso

Eng. Software / PSPD
Prof. Fernando W Cruz

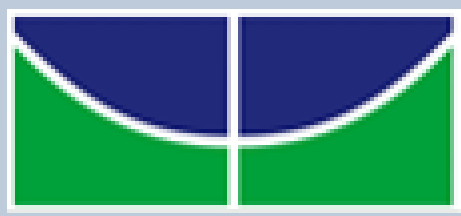


Sumário

- Sistemas de gestão centralizada/descentralizada
- Exemplos de bibliotecas e serviços de programação paralela e distribuída

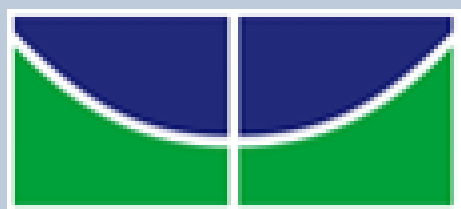


Histórico e visão geral



Descentralização: Internet

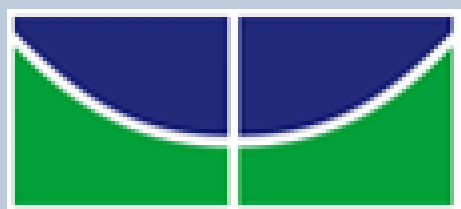
- Internet: concebida para ser descentralizada!
 - Motivação: **resiliência**, até contra uma guerra nuclear
- Projeto inclui como características:
 - **Nenhum ponto central** de controle.
 - Comutação de **pacotes**: pequenos pedaços de dados **podem seguir por caminhos diferentes** (e.g., BGP)
 - Queda de um nó **não implica em parada total** da rede.
 - Interoperabilidade, mesmo entre **dispositivos heterogêneos**
- Primórdios: discagem direta (**dial-up**)
 - Conectar ao servidor = ligar para o número correspondente
 - Bastava a rede telefônica, sem a necessidade de um provedor de Internet (*Internet Service Provider* – ISP)



Des-Descentralização da Internet

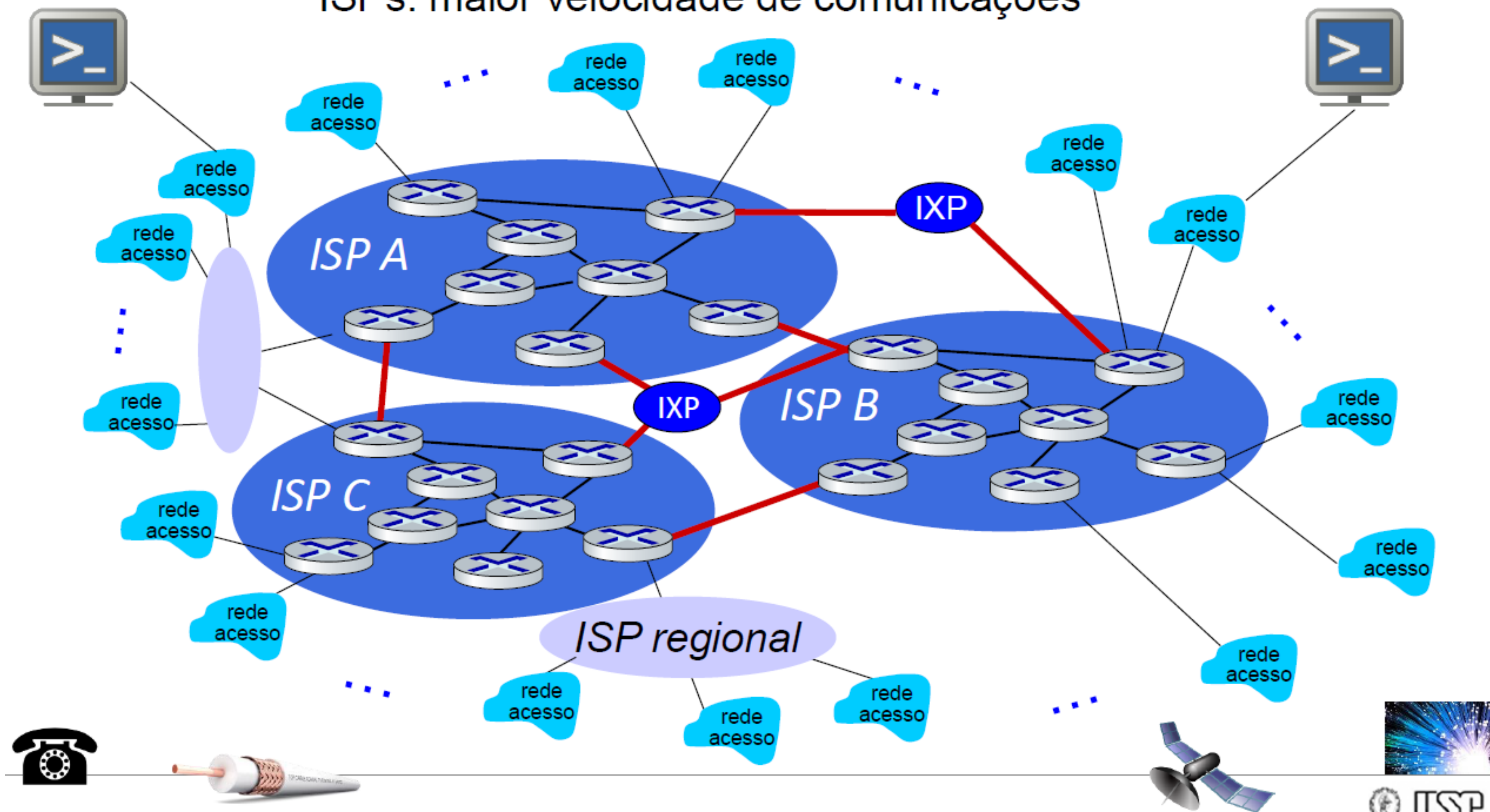
Internet: primórdios

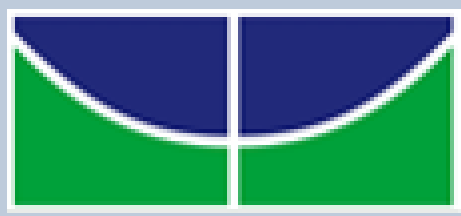




Des-Descentralização da Internet

Internet: hoje
ISPs: maior velocidade de comunicações

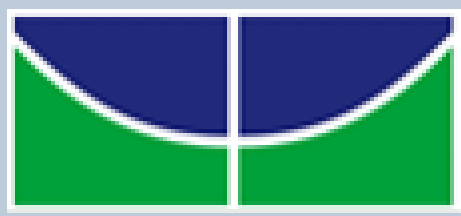




Internet: Serviços Centralizados

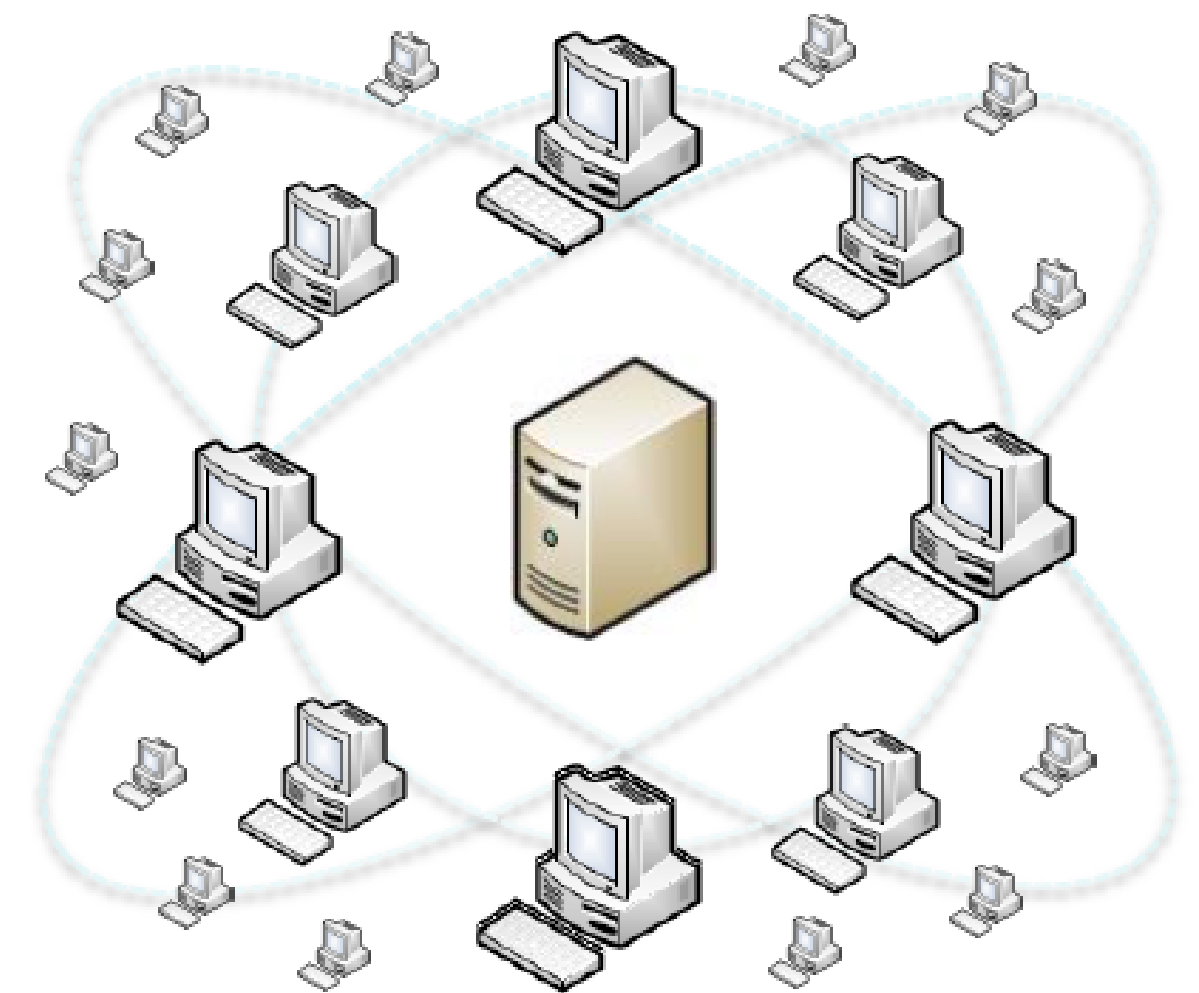
- Arquitetura principal: **cliente-servidor**
- **Cliente:**
 - Inicia comunicação com servidor (“fala primeiro”)
 - Tipicamente, solicita serviços a servidor e aguarda resposta.
 - Web: cliente implementado no browser; e-mail: leitor de correio
- **Servidor:**
 - Aguarda solicitações de clientes
 - Fornece os serviços solicitados aos clientes interessados
 - Ex.: servidor web envia a página Web solicitada; servidor de e-mail envia as mensagens, etc.



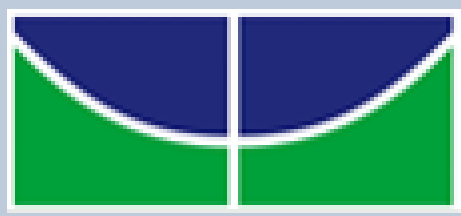


Internet: Serviços Centralizados

- **Acesso a informações e arquivos:** espalhados em vários sites...
 - Porém, busca por dados centralizada em gigantes como **Google**
- **Entretenimento:** gigantes de streaming, como **Youtube e Netflix**
- **Comunicação:** gigantes de redes sociais (**Twitter, Facebook, Instagram**) e apps de mensagem (**WhatsApp, Telegram**)
- **Operações financeiras:** bancos (Bradesco, Itau, BB, ...) e **operadoras de cartão** (Visa, Mastercard, Elo, ...)
- ...

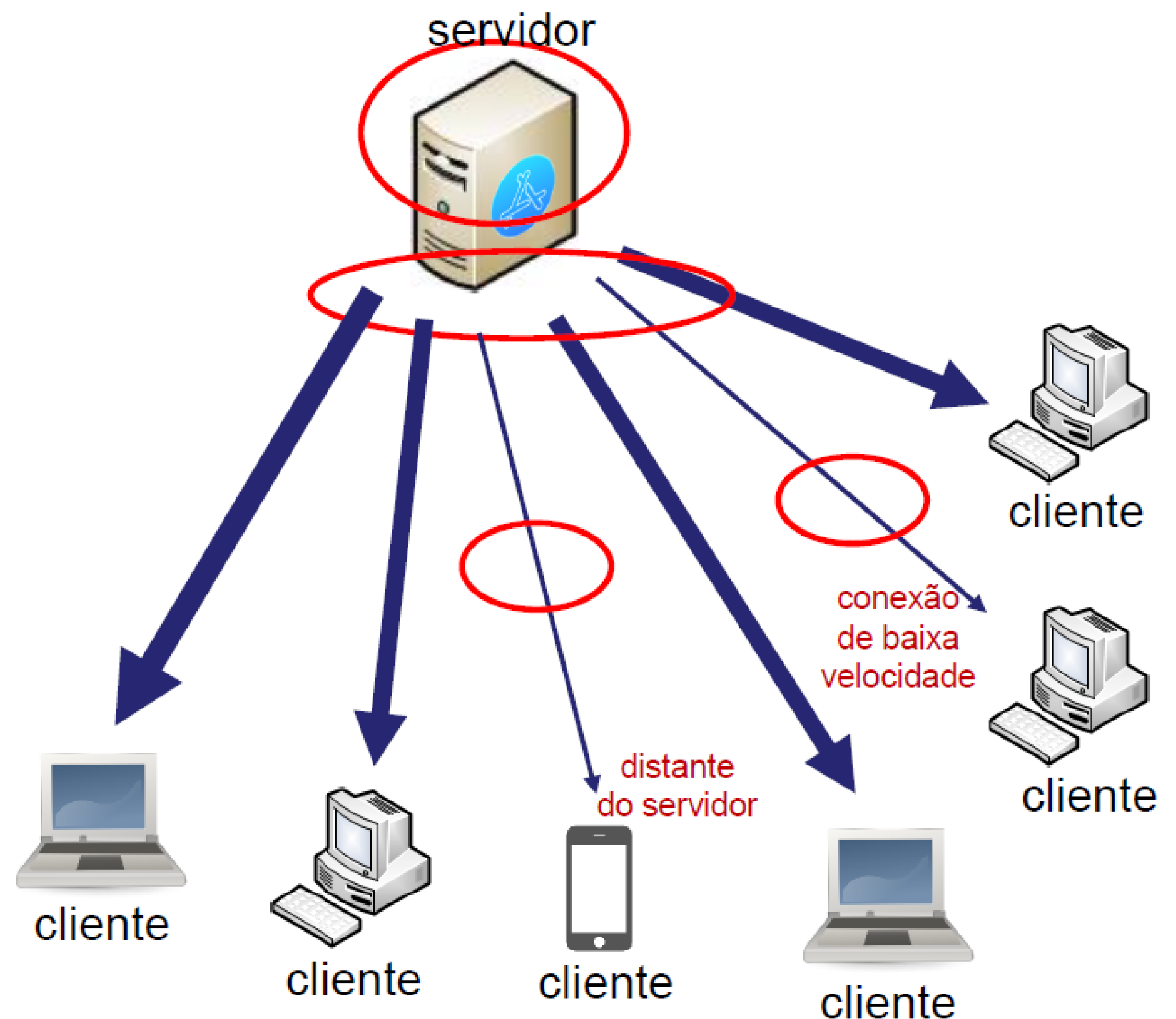


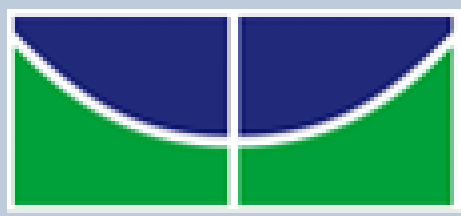
Problemas com centralização?



Serviços Centralizados: Gargalos

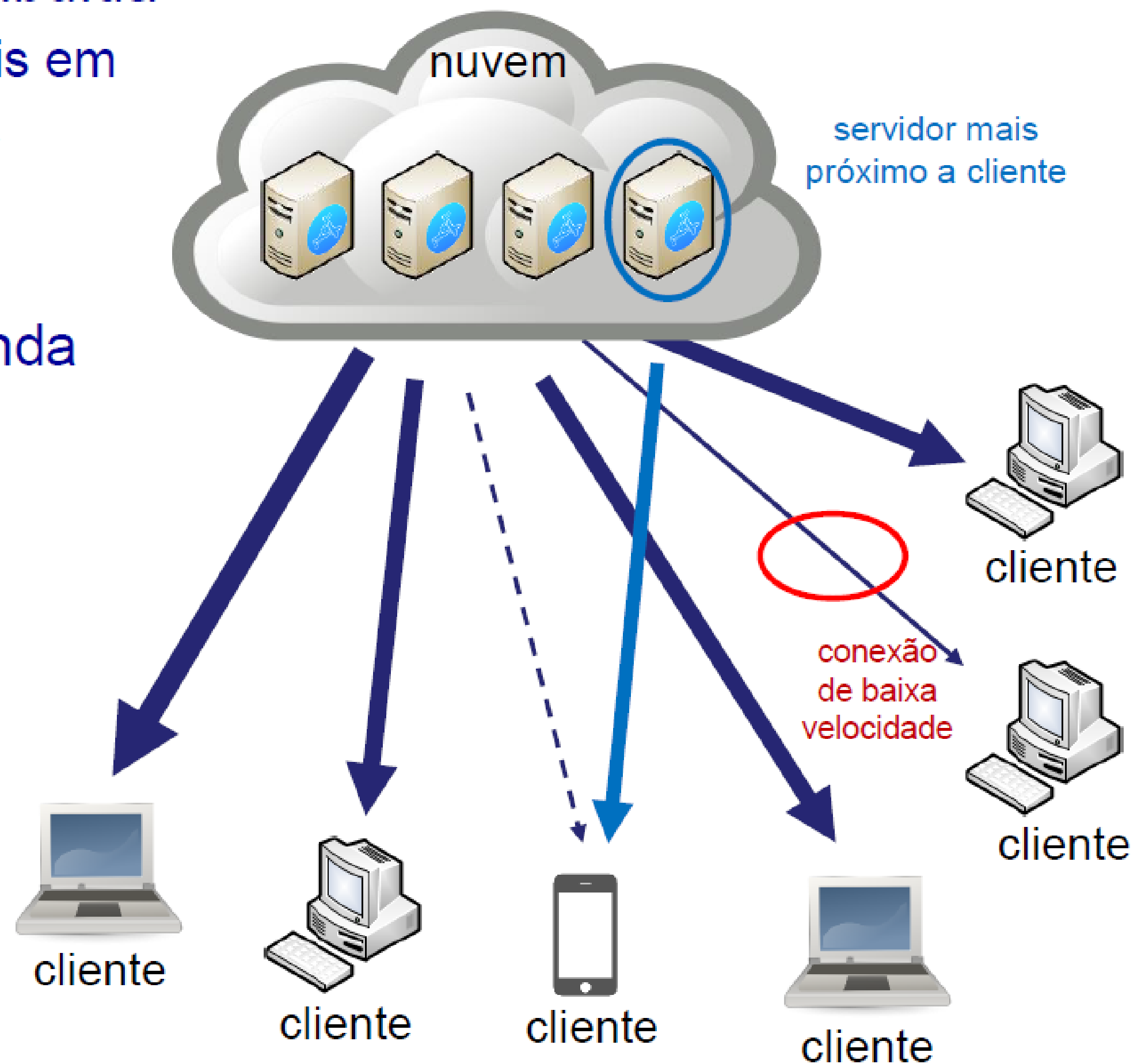
- Gargalos:
 - Servidor
 - Conexões do servidor
 - Conexões dos clientes
- Escalabilidade limitada
- Ponto único de falha
- Concentração de poder: controle sobre dados e regras do sistema

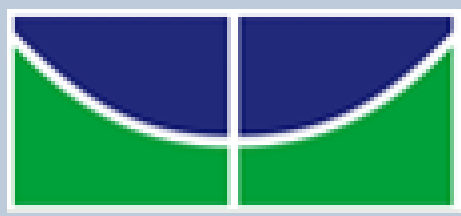




Serviços Centralizados: Gargalos

- **Nuvem:** implantação distribuída
 - **Lógica:** máquinas virtuais em balanceamento de carga
 - **Física:** datacenters em diferentes localidades
- Escalabilidade sob demanda
 - Custos do provedor
- Maior resiliência a falhas
 - Relegada a provedor
- **Concentração de poder:** provedor do serviço e da nuvem
 - controle sobre dados e regras do sistema





Serviços Centralizados: Gargalos

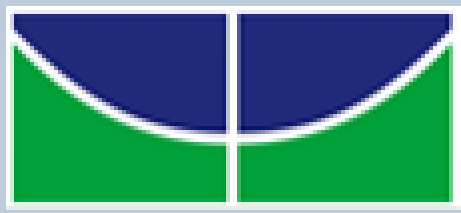
- Arquitetura cliente-servidor: cara de criar e manter
 - Custo estimado de centro de dados Google em Dalles: \$1.2bi ¹
 - \$7 bi anunciados em 2021 para construção de novos centros de dados ²
 - Cada centro usa de 50 a 100 MW de potência ³



1. "Google Data Center FAQ, Part 2" - <https://www.datacenterknowledge.com/google-data-center-faq-part-2>

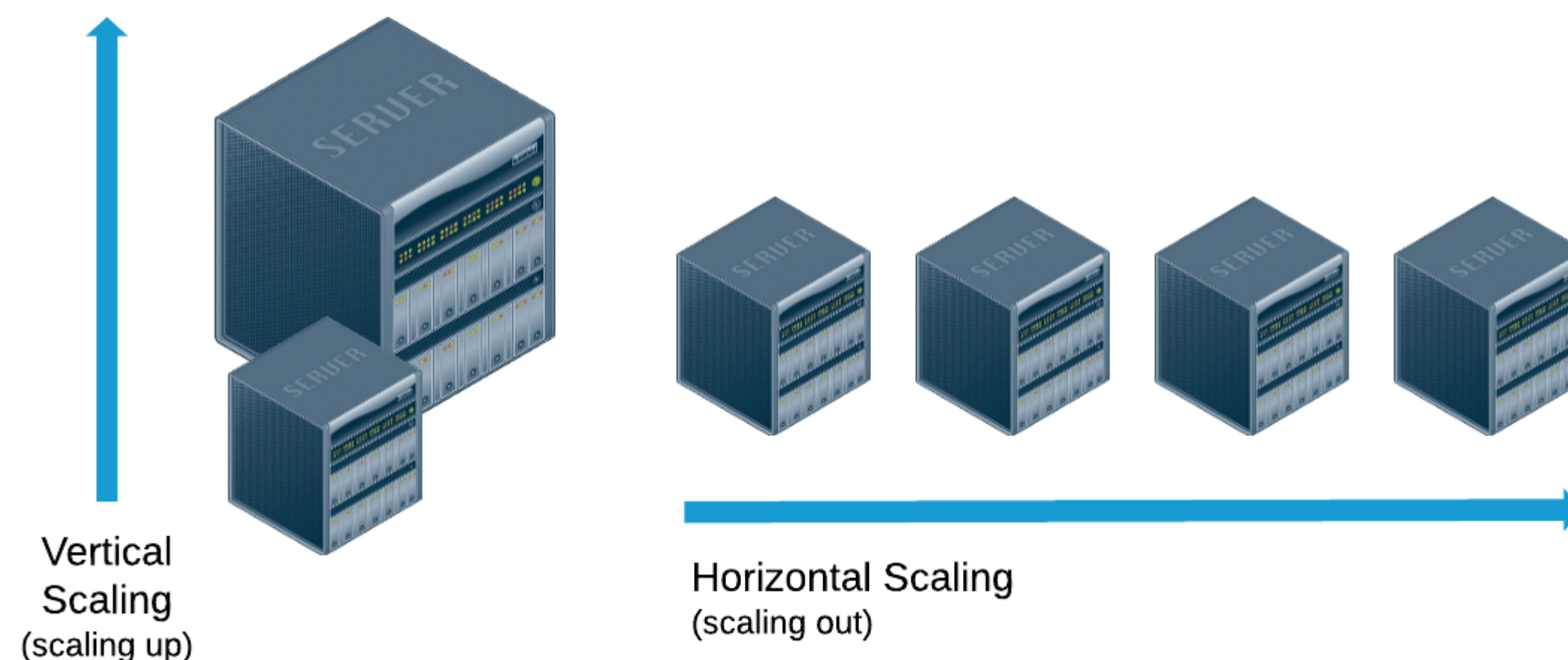
2. "Google outlines \$7B U.S. data center, office investment" <https://www.fiercetelecom.com/telecom/google-outlines-7b-u-s-data-center-office-investment>

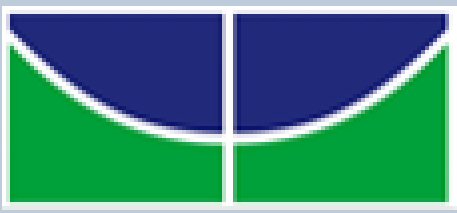
3. "Baxtel - Google Locations" <https://baxtel.com/data-centers/google>



Frameworks de computação paralela

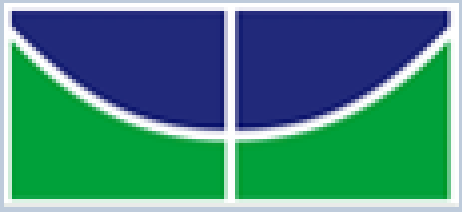
- Explora o scale-out
- Cloud computing
- Big Data
- Mudança no estilo de programação ...
- ... e no foco das aplicações
- E novas profissões mais voltadas para dados do que para o código (Cientista/Engenheiro de dados)





Volume de dados gerados

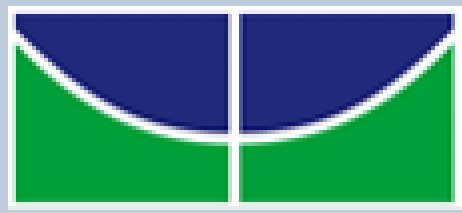
- Facebook - recebe 1 bilhão de usuários por dia. Mais de 4 bilhões de likes e 300 milhões
- Netflix tem 75 milhões de assinantes e 15 mil títulos. 30 por cento de toda a banda dos EUA
 - Novos conteúdos em função do entendimento desses usuários
- Existem mais dispositivos conectados na Internet do que seres humanos (em torno de 20 bilhões de dispositivos em 2020)... Cada dispositivo gerando muitos gigabytes de dados.
- O LHC produz um petabyte de dados por segundo. Mas apenas 1% dos dados são processados gerando 25 petabytes.



Supercomputadores – anos 60



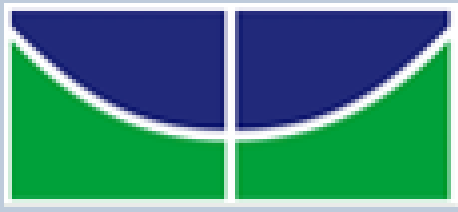
http://upload.wikimedia.org/wikipedia/commons/c/c4/CDC_6600.jc.jpg



Clusters – década de 1970

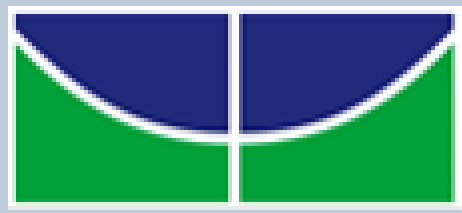


<http://upload.wikimedia.org/wikipedia/commons/3/3d/Us-nasa-columbia.jpg>



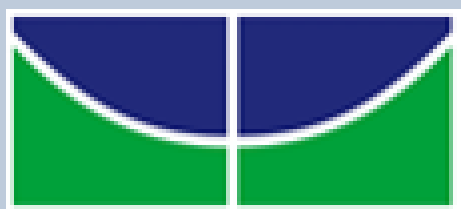
Grids – desde a década de 1990





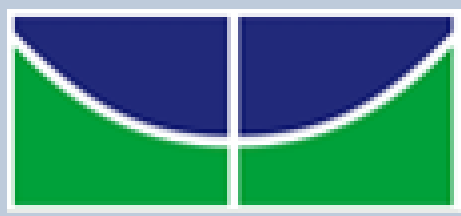
Sala de racks/servidores (cluster)





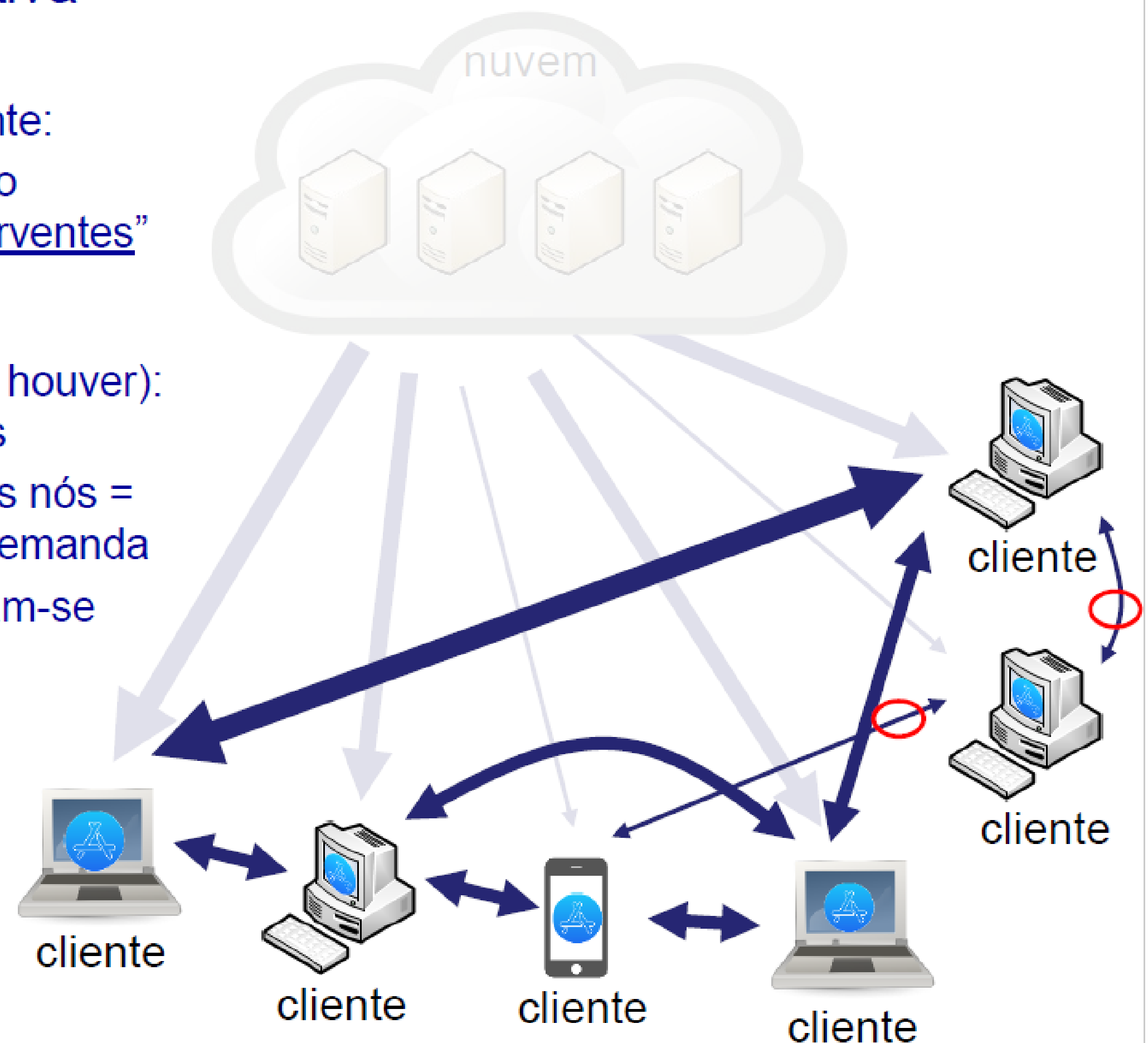
Frameworks a serem estudados

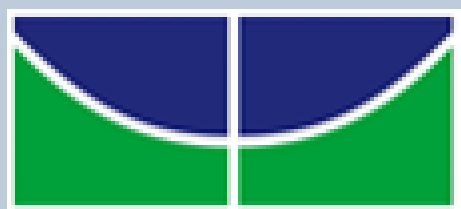




Serviços Descentralizados

- Alternativa mais colaborativa (*peer-to-peer*. P2P):
 - Nós se comunicam diretamente:
 - Atuam simultaneamente como servidores e clientes: são “serventes”
- Benefícios:
 - **Alivia carga no servidor** (se houver): explora conexões alternativas
 - **Escalável por natureza**: mais nós = mais recursos, não só mais demanda
 - **Resiliência inerente**: eliminam-se pontos centrais de falha
 - **Custos** de operação distribuídos entre nós
 - **Controle distribuído**: os usuários são o sistema
 - Em alguns casos, maior **privacidade**





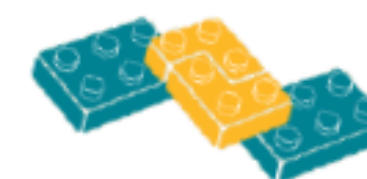
Serviços Descentralizados: Desafios

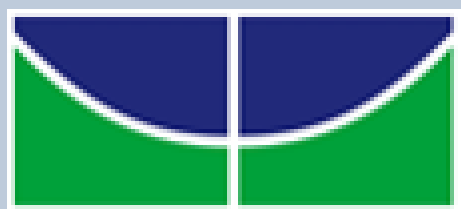
- P2P: diversos atrativos, mas nem tudo são flores...



- **Desafios de administração**

- **Entrada e saída dos nós dinamicamente**
 - Solução: **redundância**; detecção & recuperação
- **Difícil garantir qualidade de serviço**
 - Solução: **incentivo** para colaboração
- **Heterogeneidade dos nós:**
 - Solução: **middlewares** para abstração; padrões **abertos**
- **Acesso concorrente, sem relógio global: possíveis conflitos**
 - Solução: **consistência eventual**; mecanismos de **consenso**
- **Baixa confiança nos nós participantes**
 - Solução: criptografia e protocolos de segurança
- **Localização de nós e recursos distribuídos na rede**
 - Solução: mecanismos de busca distribuídos; broadcast de dados

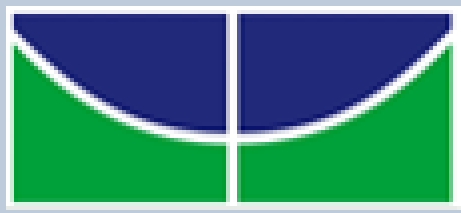




Serviços descentralizados: novidade?

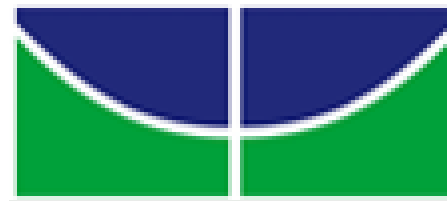
- Popularização de blockchains elevou interesse em iniciativas de “descentralização”
 - “DApp”: decentralized application
- Porém, vários “DApps” notórios surgiram bem antes (e não dependem de) blockchains
 -  **napster**™ (1999): compartilhamento de músicas
 -  (1999): busca de vida extraterrestre
 -  **BitTorrent**™ (2001): compartilhamento de arquivos
- No final, tecnologias P2P retornam Internet a sua visão original: todos criam e consomem conteúdo
 - E um toque de “anarquia”: colaboração direta, liberdade, resistência a censura, robustez, confiabilidade, ...



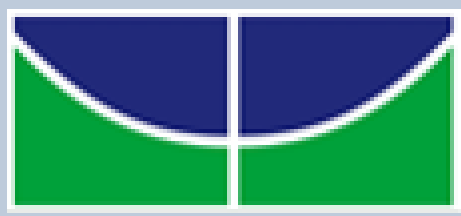


Alguns temas de estudo na descentralização

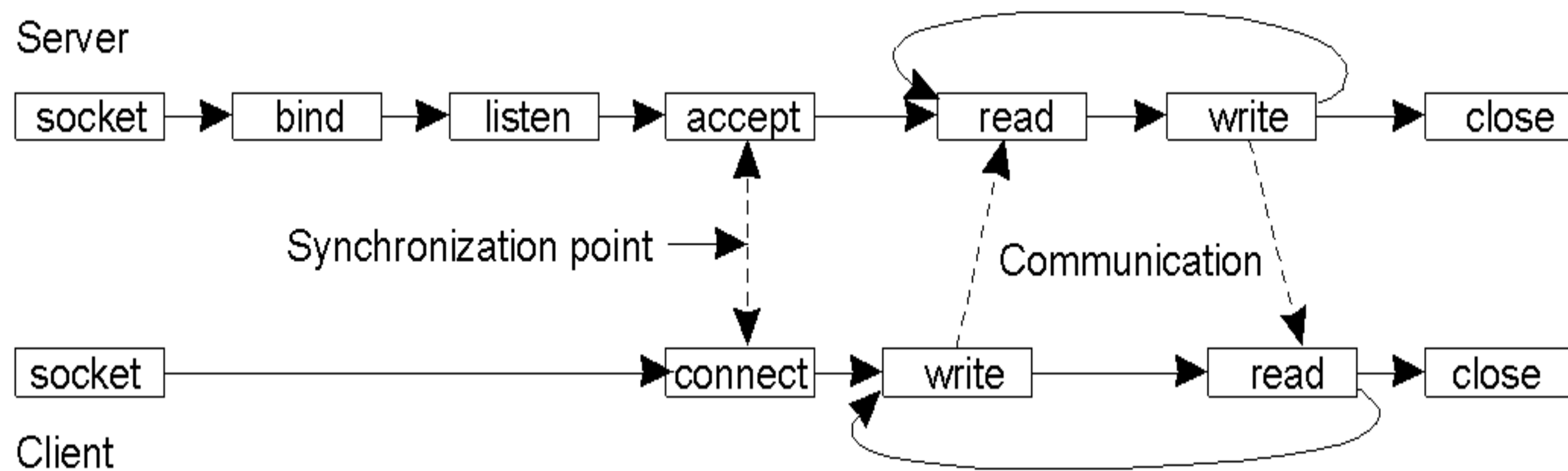
- P2P
- Blockchain
- Algoritmos de consenso
- Consistência/replicação
- ... e outros temas relacionados à computação distribuída/paralela



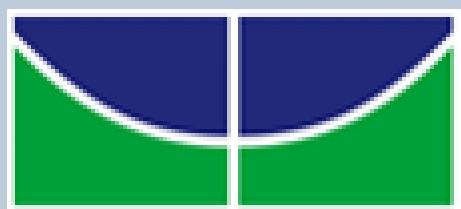
Exemplos de bibliotecas e serviços de programação paralela e distribuída



1. Berkeley Sockets



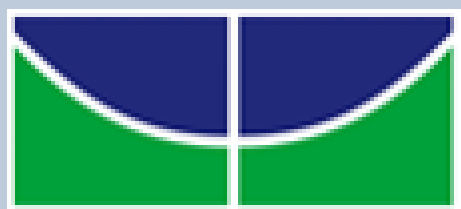
- Exemplo de comunicação orientada à conexão usando socket TCP



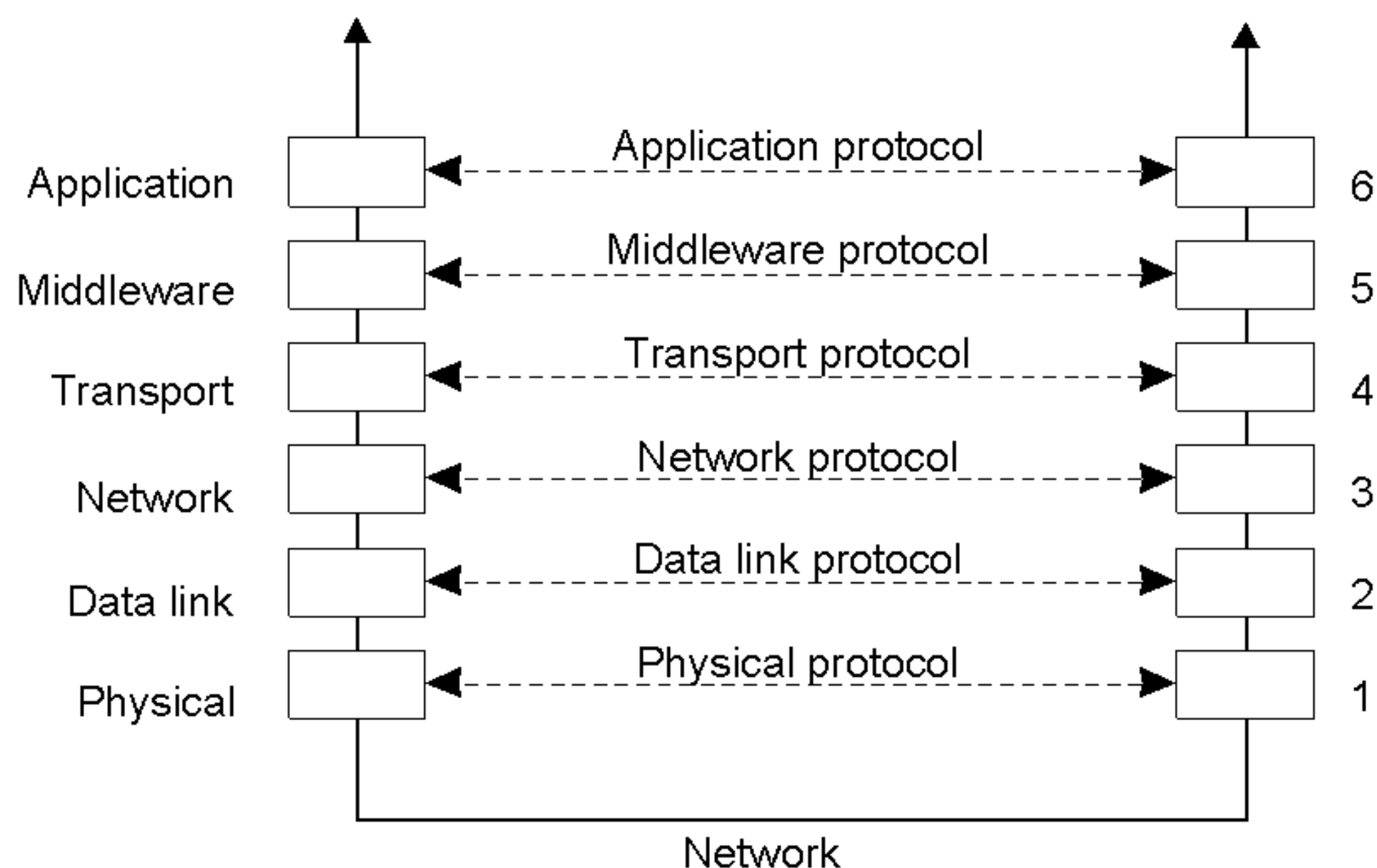
1. Berkeley Sockets

- Primitivas necessárias para programação com sockets TCP/IP

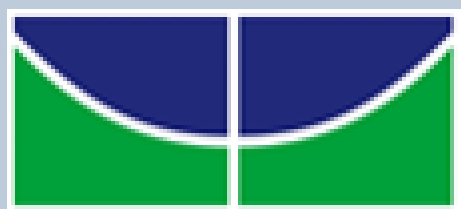
Primitive	Meaning
Socket	Create a new communication endpoint
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection



2. RPC – *Remote procedure call*

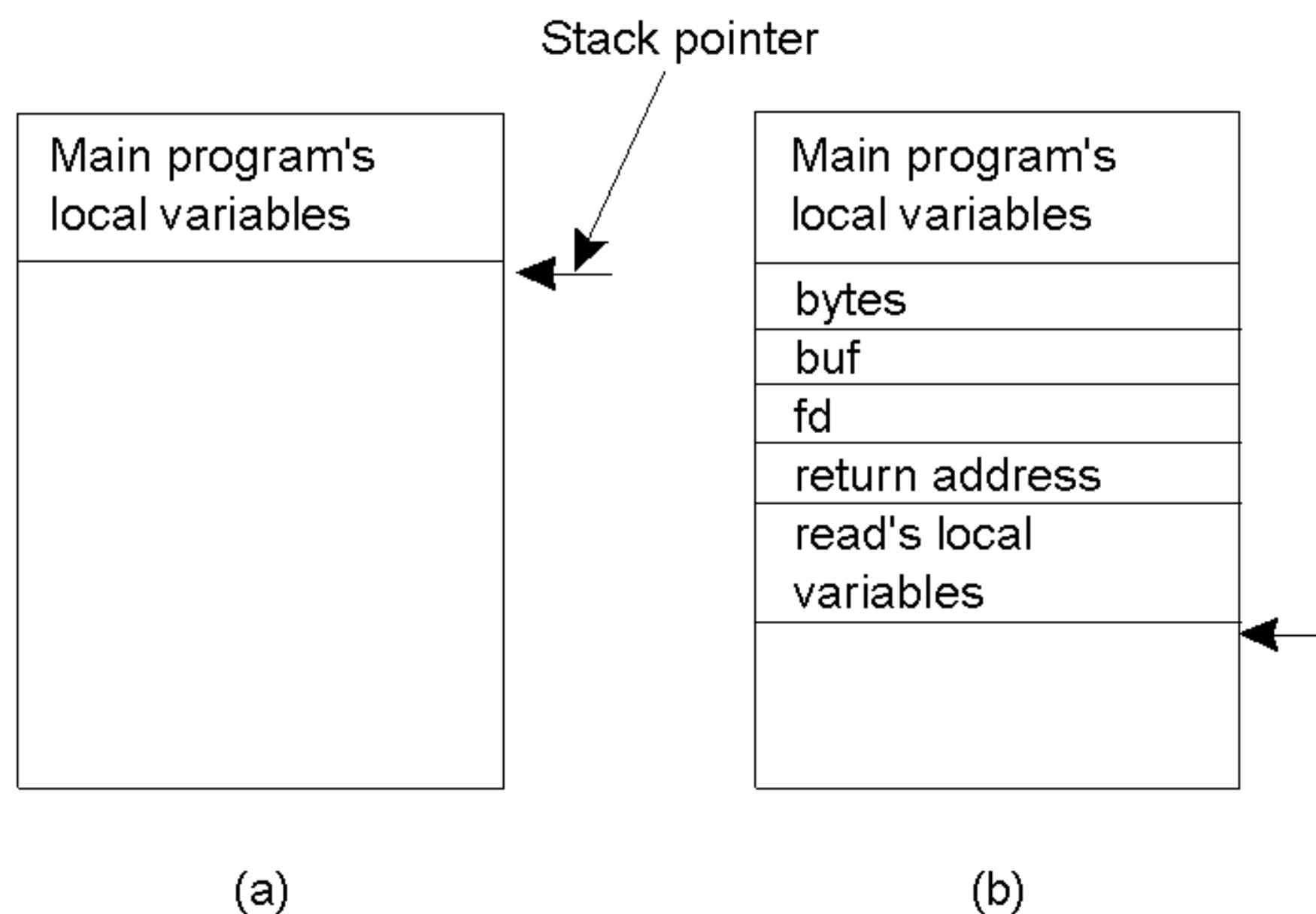


- Um modelo de referência adaptado para comunicação em rede

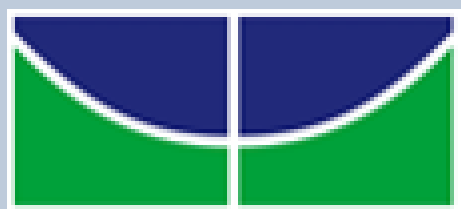


2. RPC – *Remote procedure call*

Relembrando chamadas locais



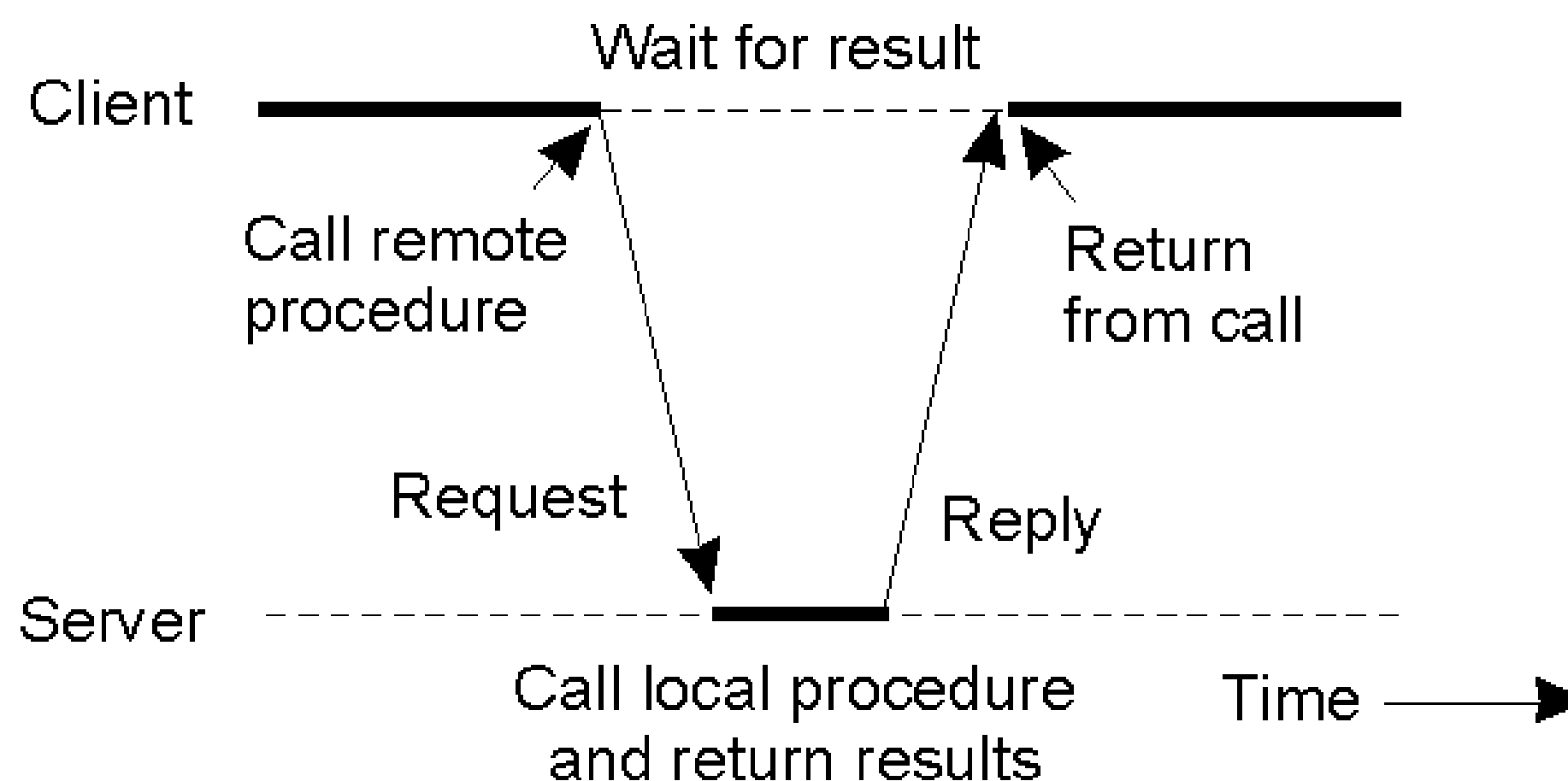
- a) Passagem de parâmetro em uma chamada de procedure local: a pilha antes da chamada
- b) A pilha enquanto a chamada de procedimento está ativa

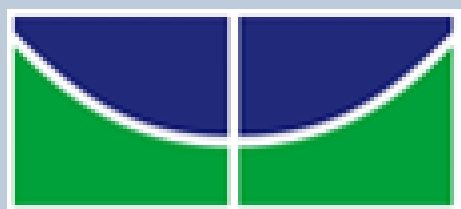


2. RPC – *Remote procedure call*

RPC envolve o conceito de stubs nos lados cliente e servidor

- Princípio do RPC entre um programa cliente-servidor

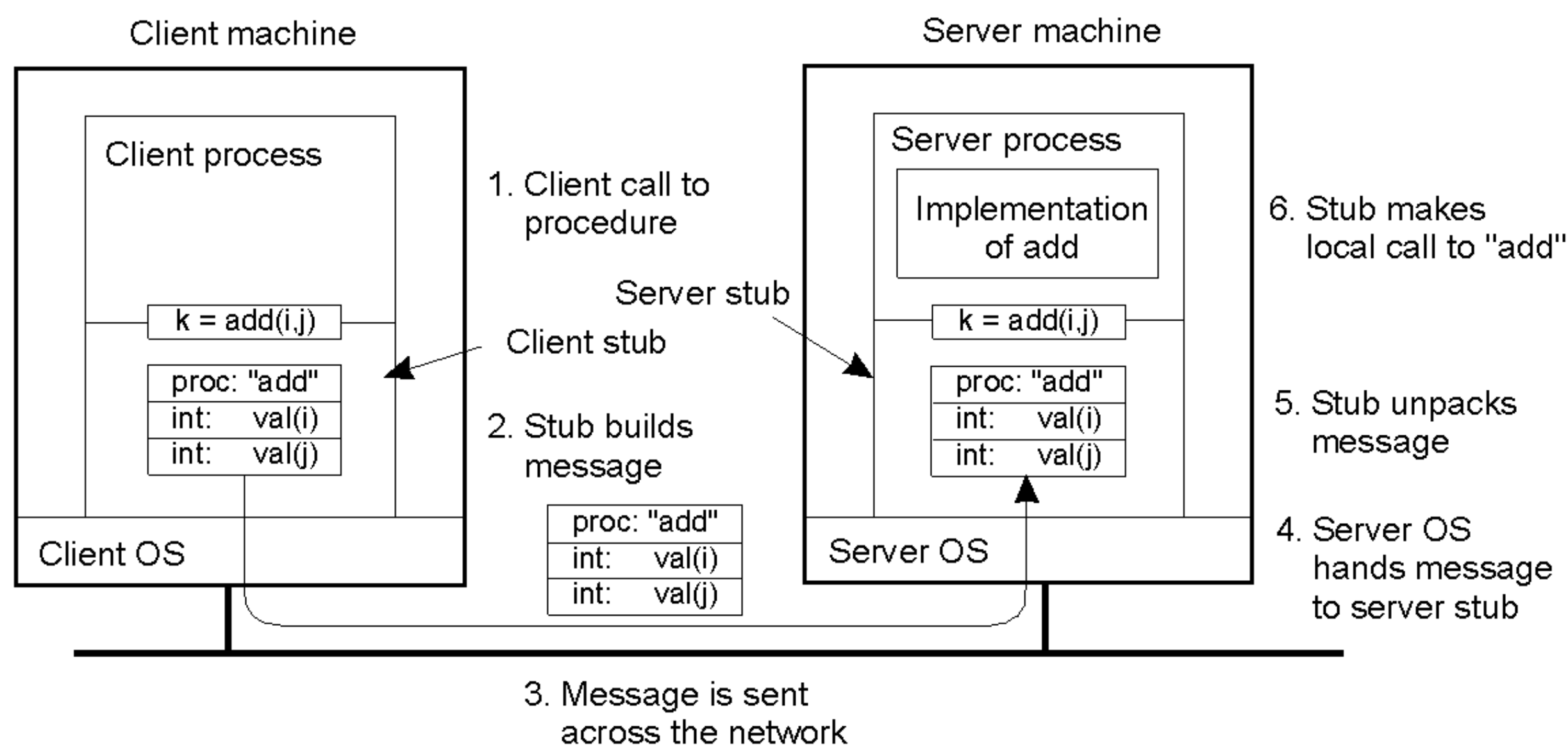


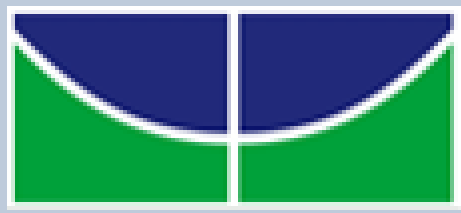


2. RPC – *Remote procedure call*

RPC cuida da passagem de parâmetros entre funções em hosts distintos

- Passos envolvidos na computação usando RPC





2. RPC – *Remote procedure call*

RPC cuida da passagem de parâmetros entre funções em hosts distintos

3	2	1	0
0	0	0	5
7	6	5	4
L	L	I	J

(a)

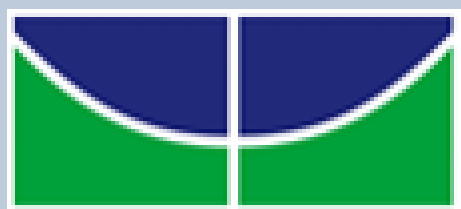
0	1	2	3
5	0	0	0
4	5	6	7
J	I	L	L

(b)

0	1	2	3
0	0	0	5
4	5	6	7
L	L	I	J

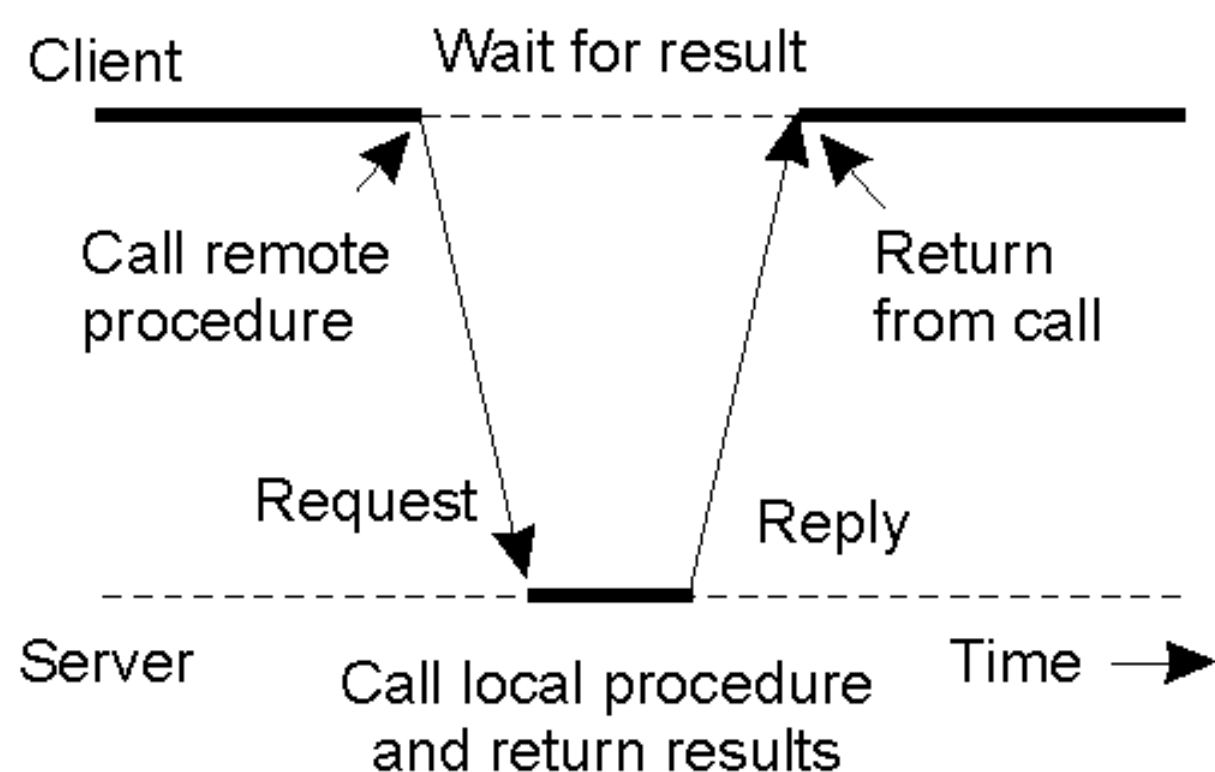
(c)

- a) Mensagem original em um Pentium
- b) Mensagem após ser recebida em um host SPARC
- c) A mensagem após ser invertida. Os números menores nas caixas indicam o endereço de cada byte

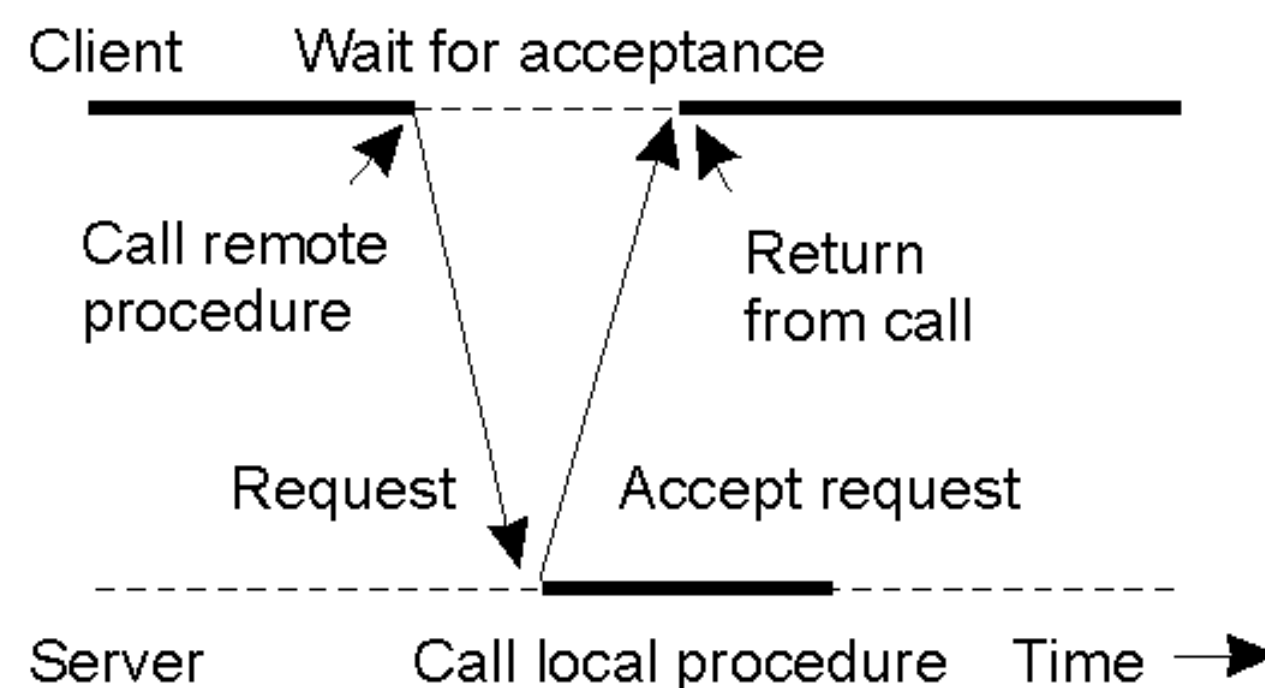


2. RPC – *Remote procedure call*

RPC permite chamadas assíncronas

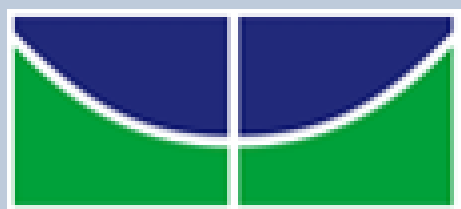


(a)



(b)

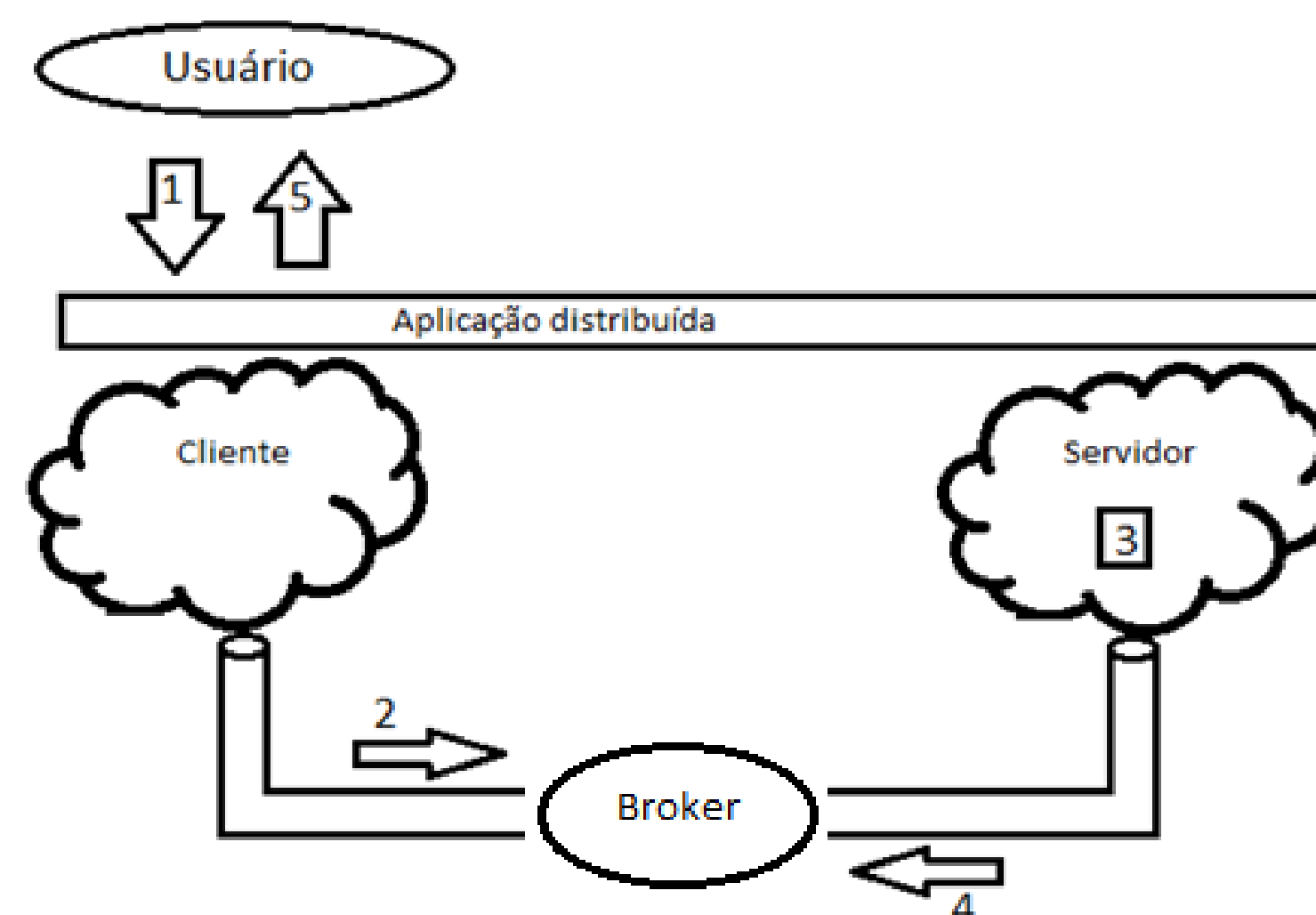
- a) A interconexão entre cliente e servidor em um RPC tradicional
- b) Interação usando RPC assíncrono



3. Uso de *brokers*

Intermediários na comunicação entre master e workers

- Paradigma publish/subscribe
- As filas de mensagens passam a ser o referencial principal
- Exemplos: RabbitMQ e Kafka



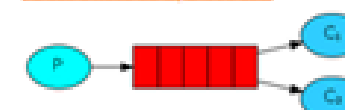
1 "Hello World!"

The simplest thing that does something



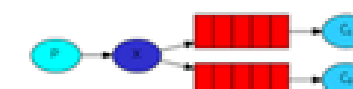
2 Work queues

Distributing tasks among workers (the competing consumers pattern)



3 Publish/Subscribe

Sending messages to many consumers at once



4 Routing

Receiving messages selectively



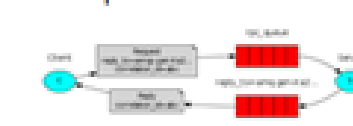
5 Topics

Receiving messages based on a pattern (topics)



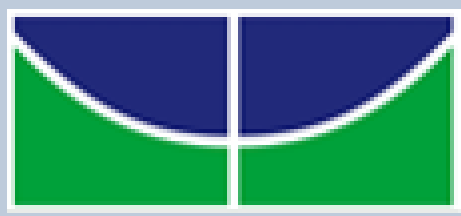
6 RPC

Request/reply pattern example

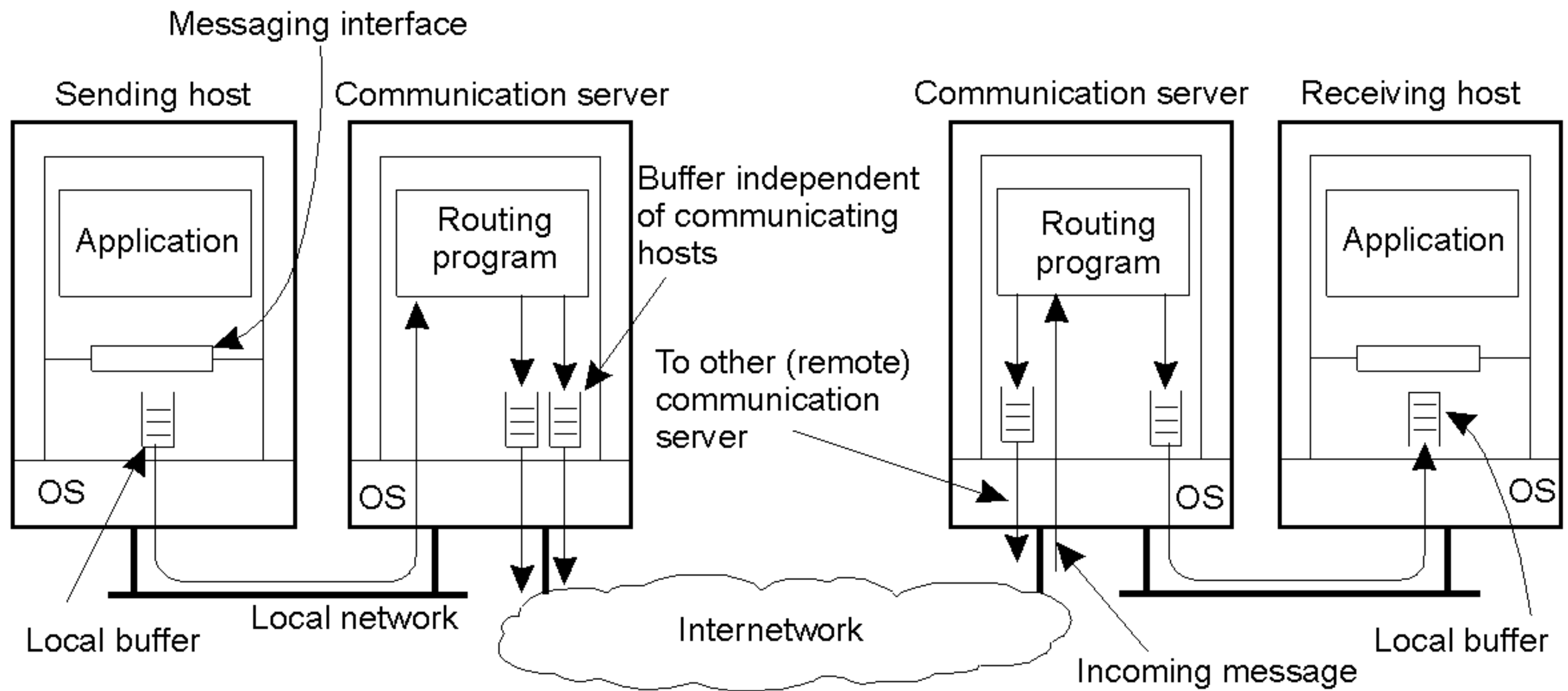


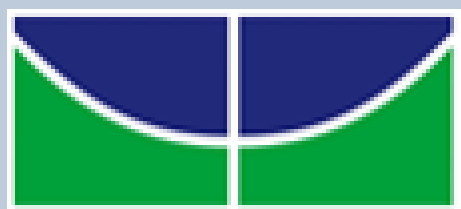
7 Publisher Confirms

Reliable publishing with publisher confirms



3. Uso de brokers

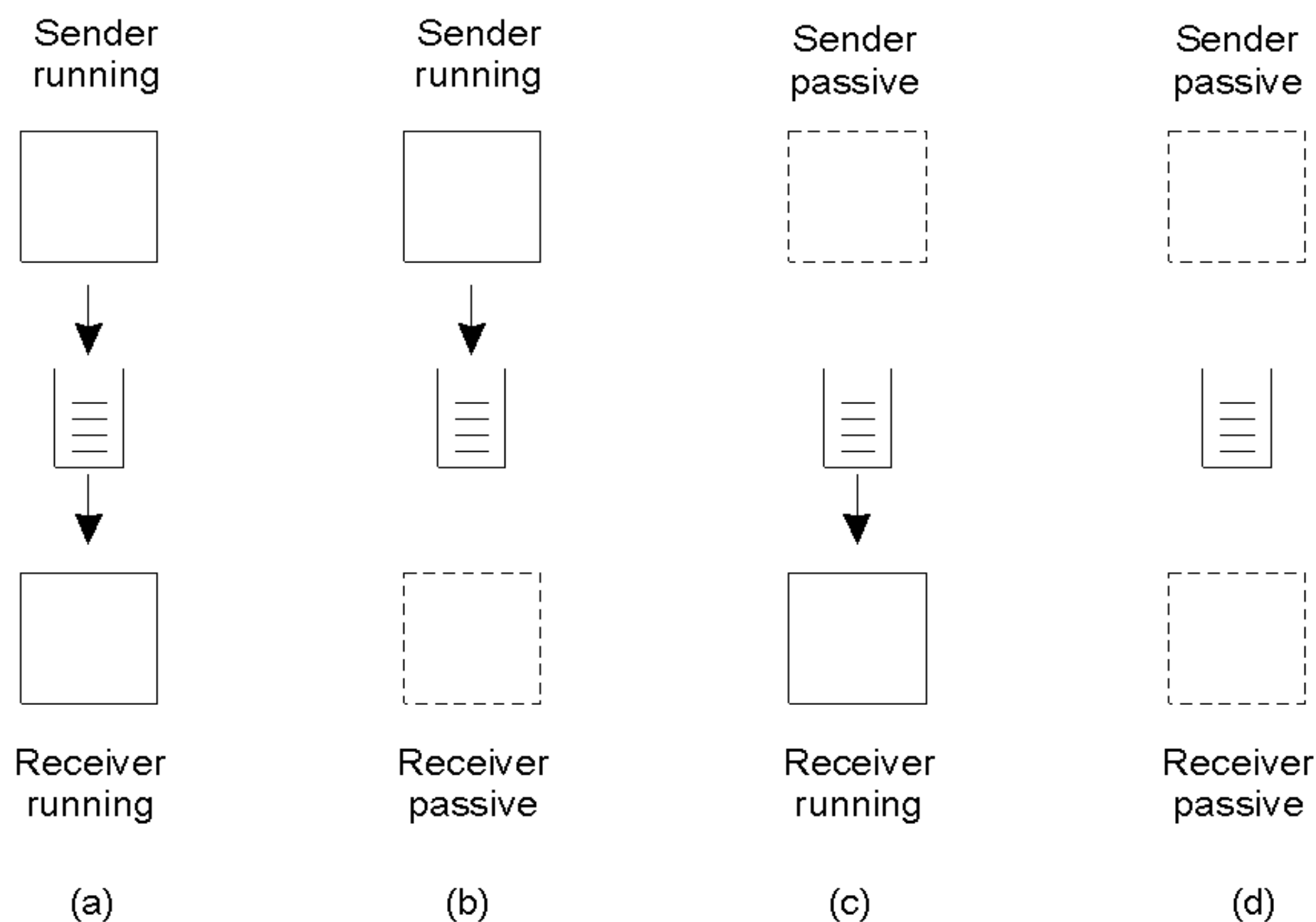
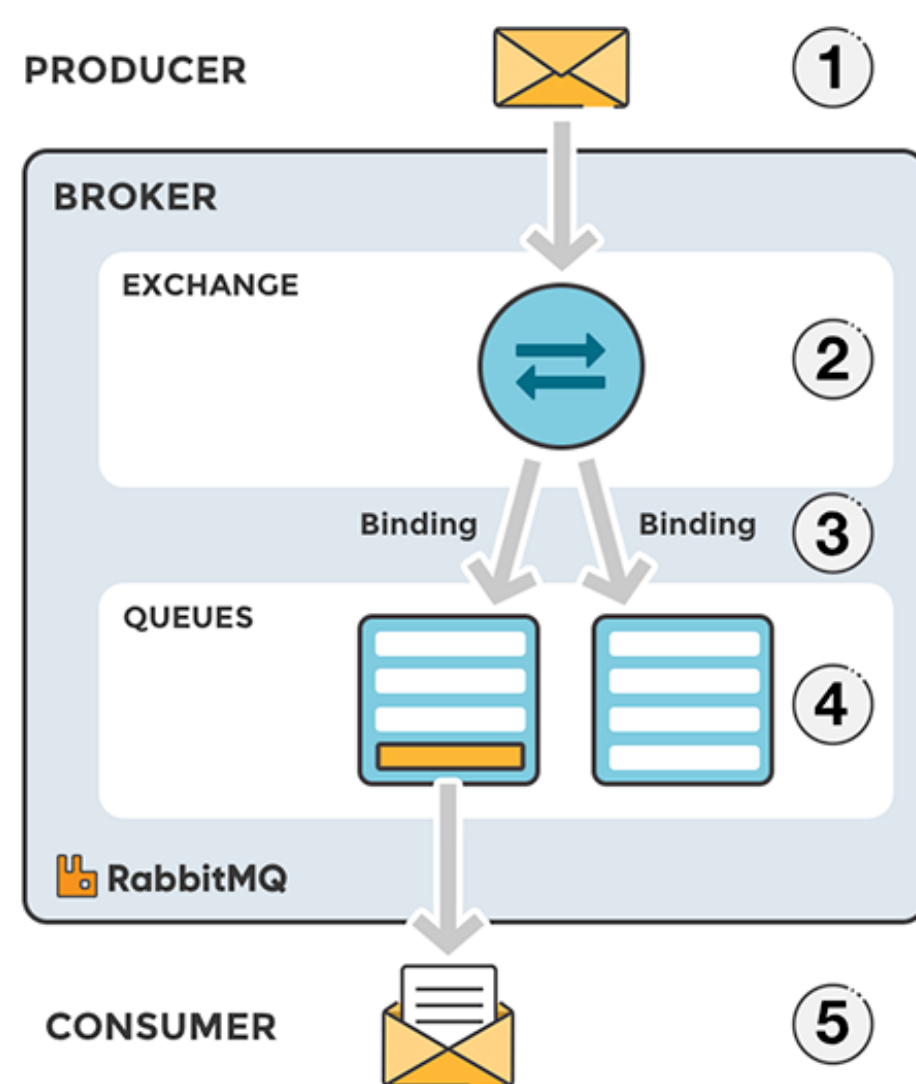


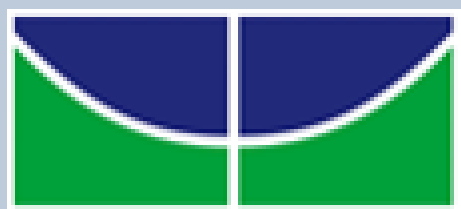


3. Brokers e filas de mensagens

Características dos modelos baseados em filas

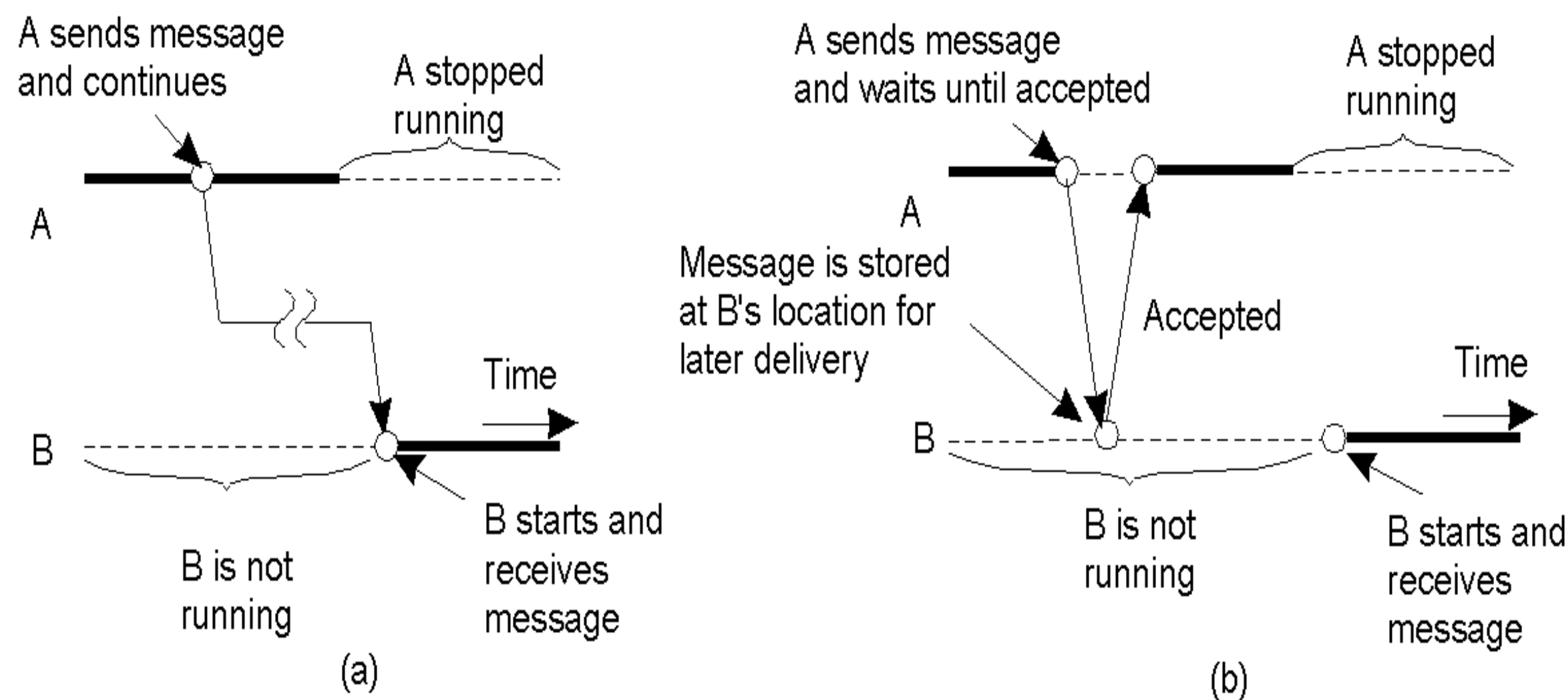
- Combinações possíveis para comunicação com fraco acoplamento



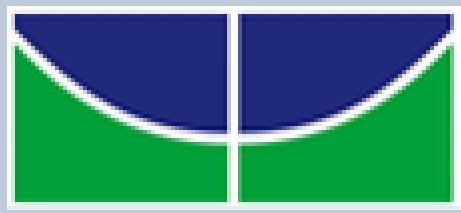


Características da comunicação

Persistência e sincronicidade em uma comunicação

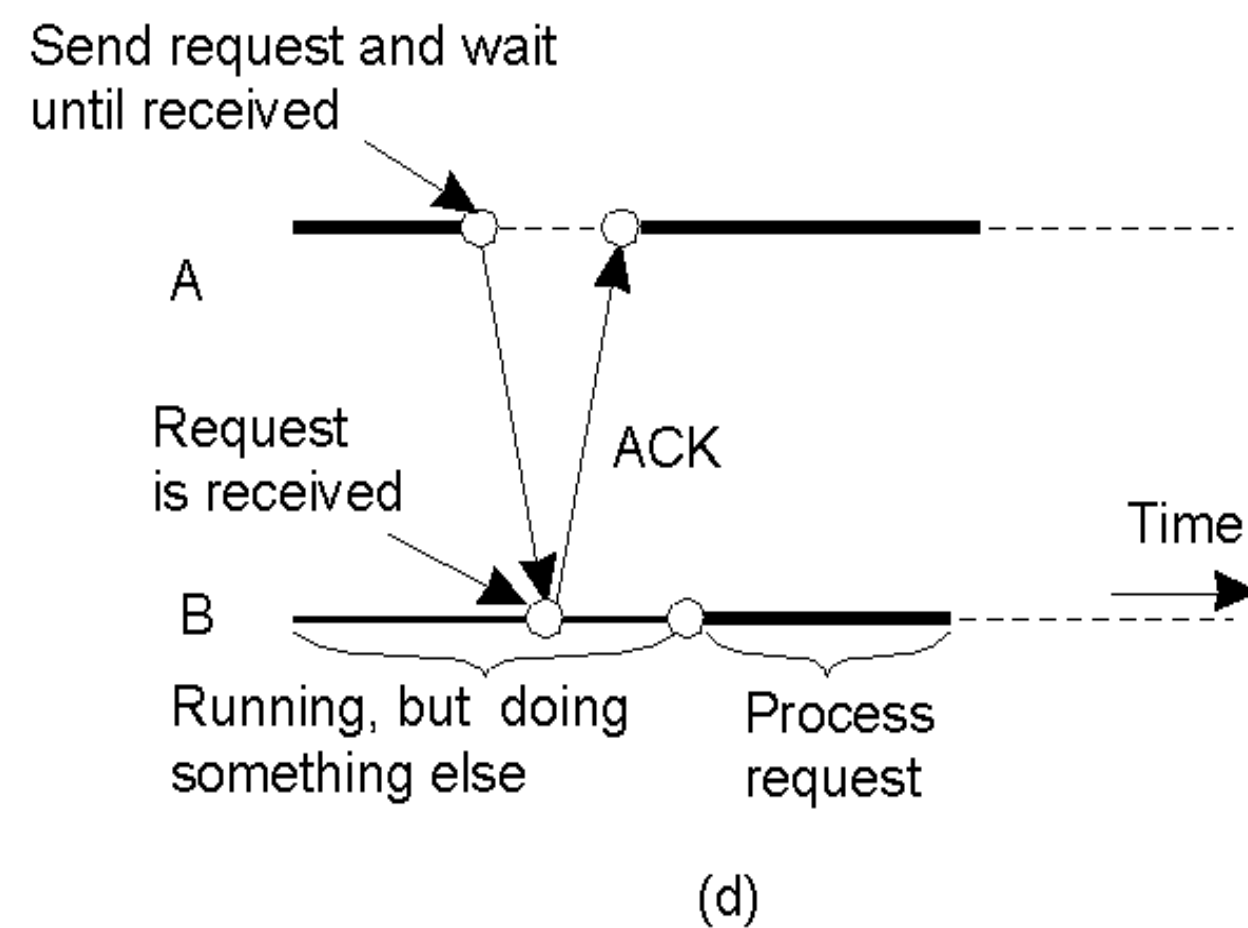
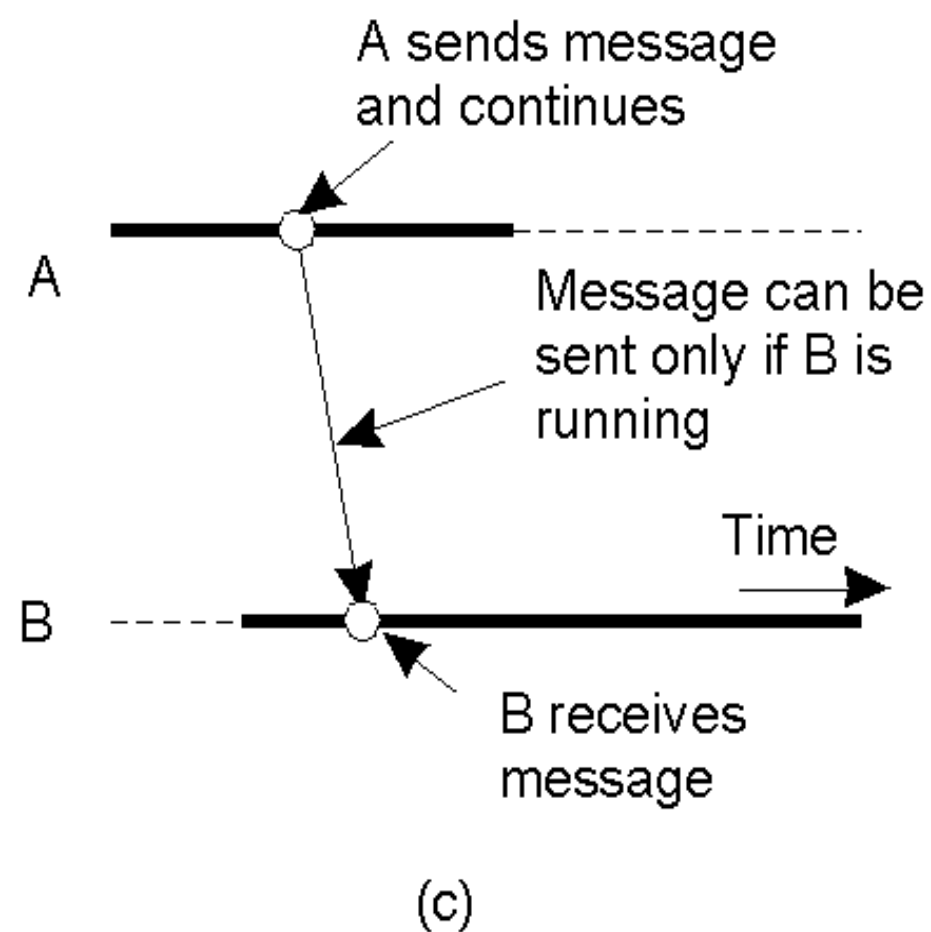


- a) Comunicação assíncrona persistente
- b) Comunicação síncrona persistente

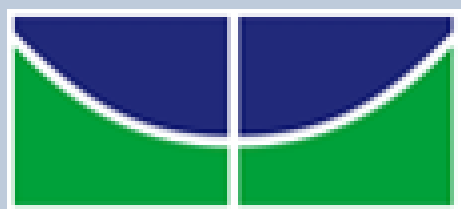


Características da comunicação

Persistência e sincronicidade em uma comunicação

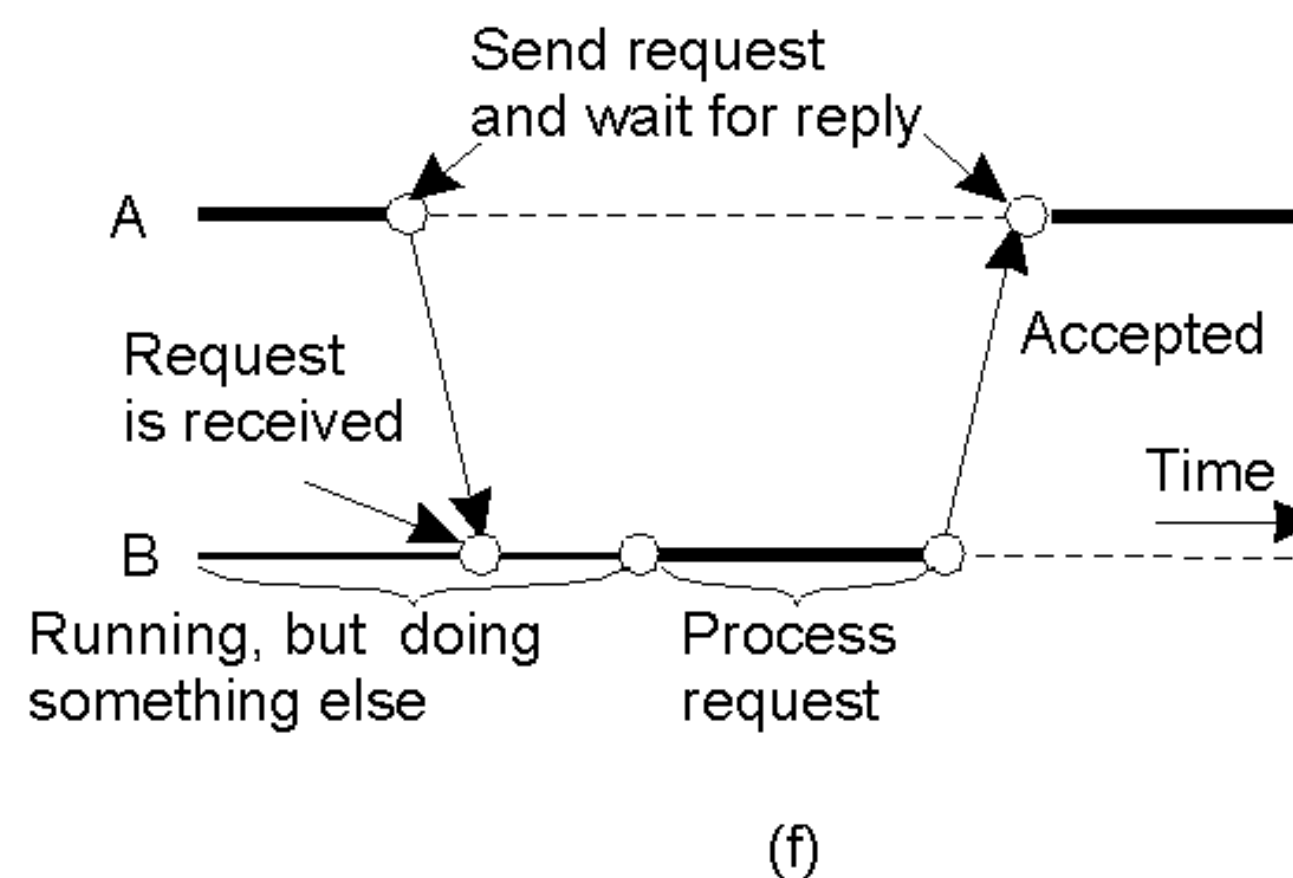
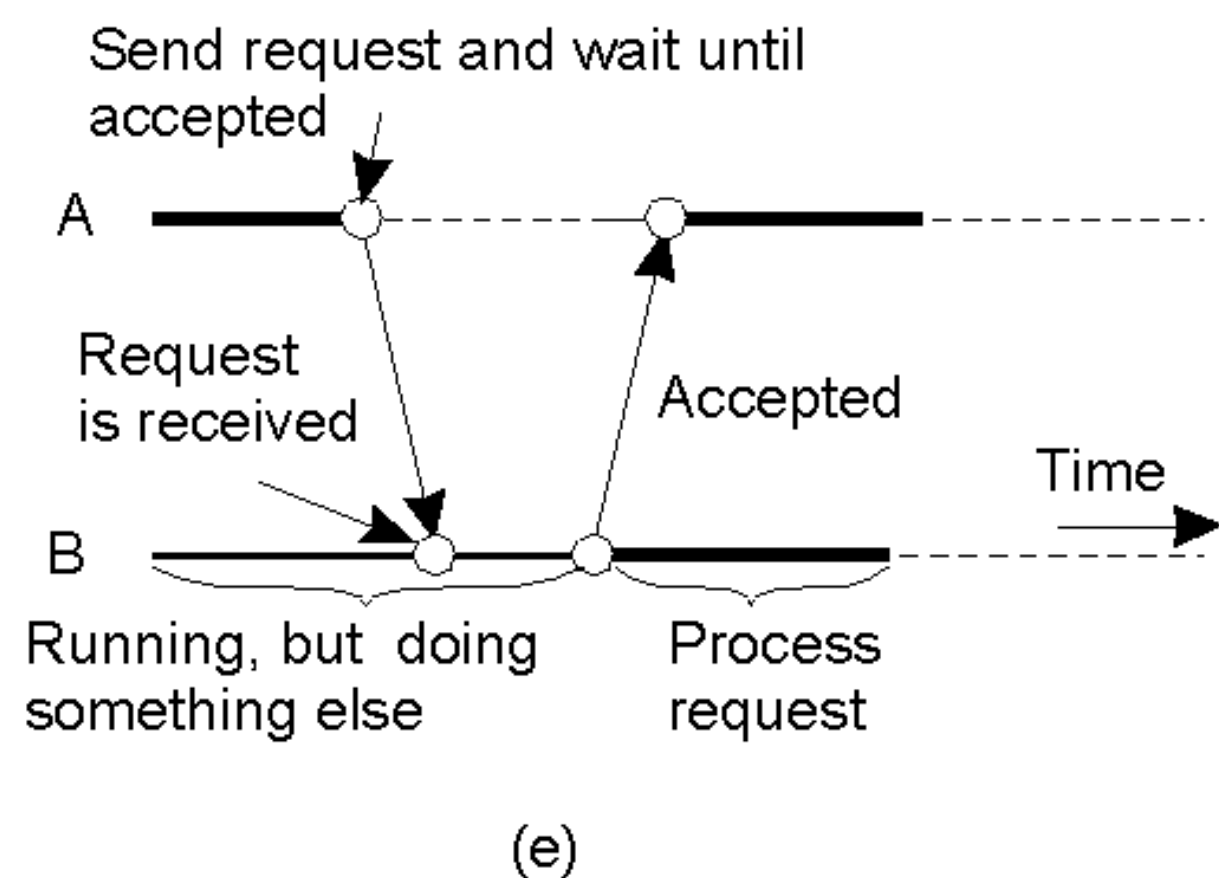


- c) Comunicação transiente assíncrona
- d) Comunicação síncrona transiente baseada no receptor

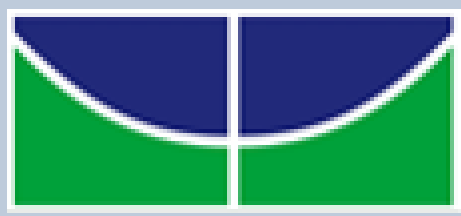


Características da comunicação

Persistence and Synchronicity in Communication (5)

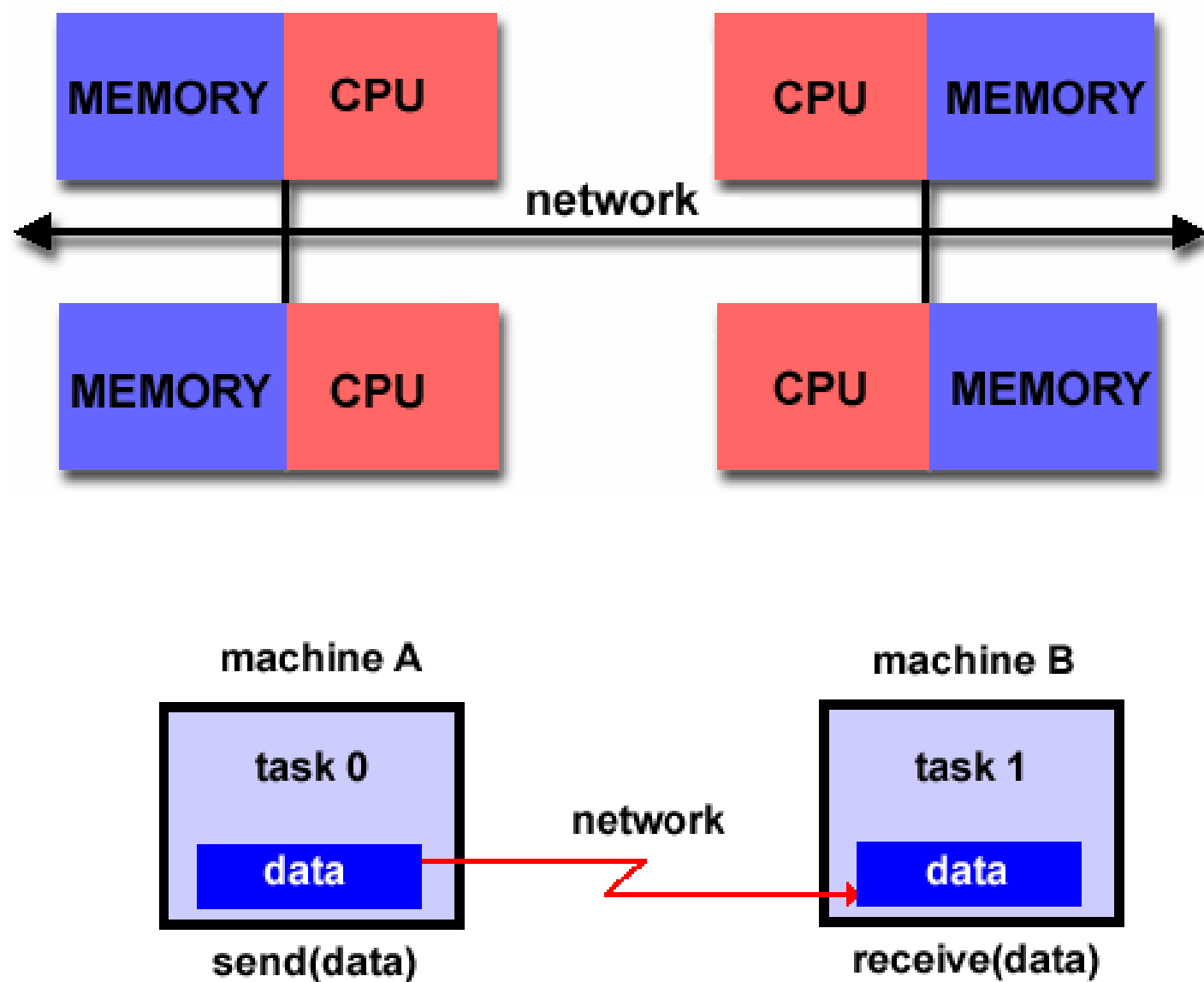


- e) Delivery-based transient synchronous communication at message delivery
- f) Response-based transient synchronous communication

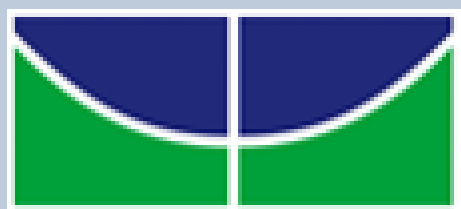


4. MPI – Message Passing Interface

- Biblioteca especial para programação distribuída (comunicação e sincronização)

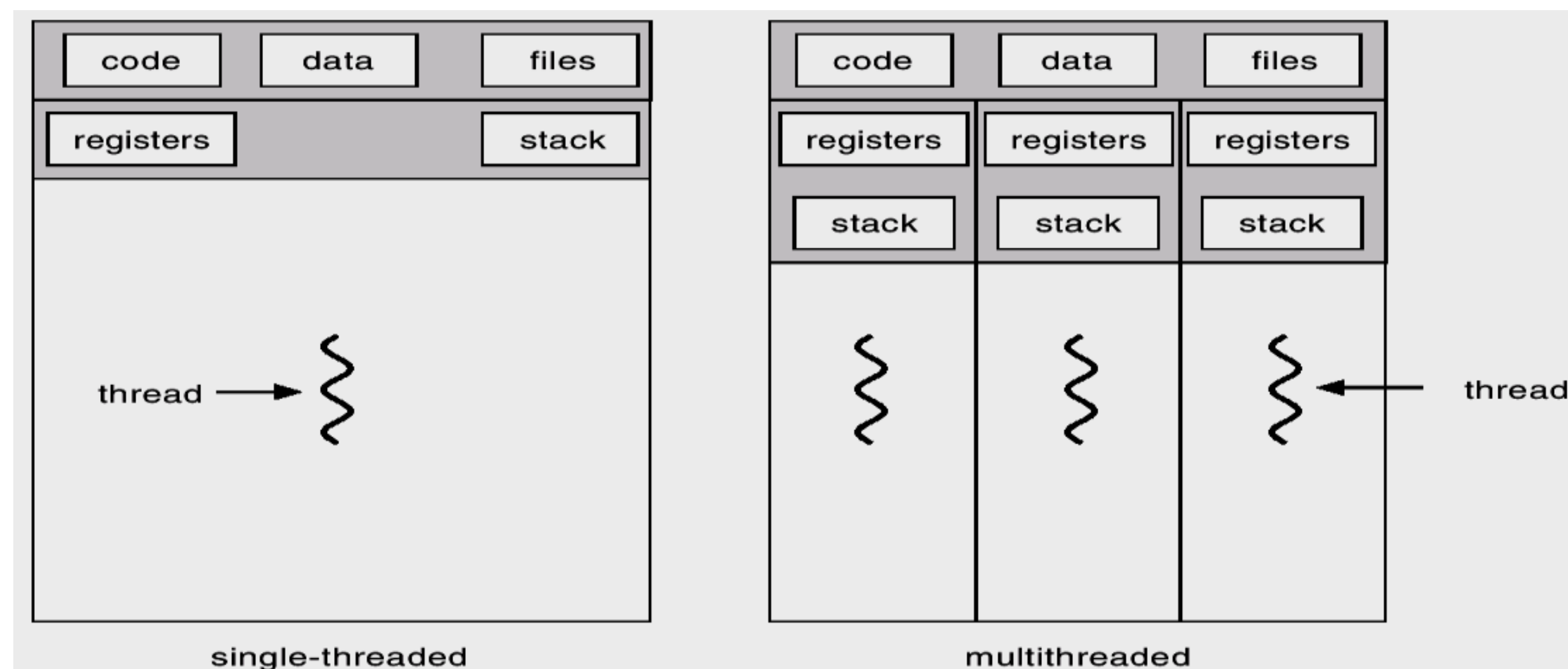
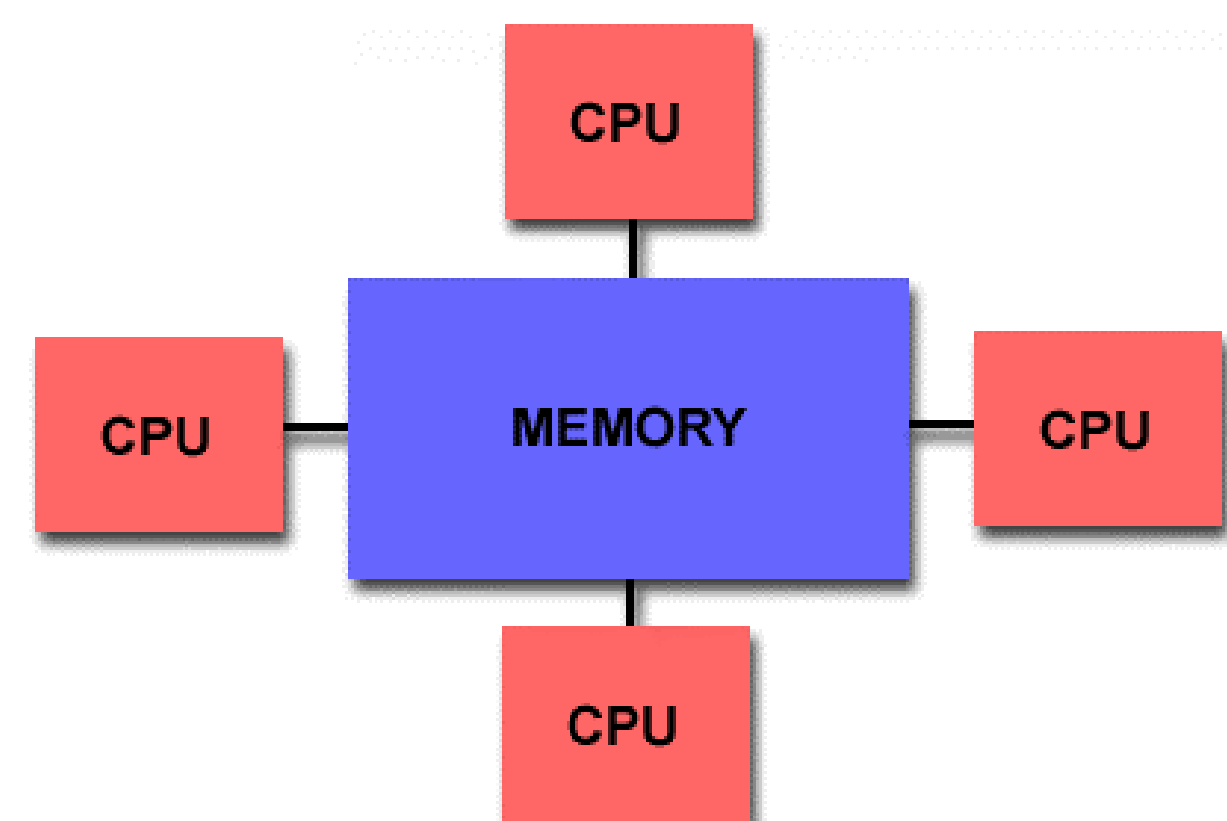


Primitive	Meaning
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until receipt starts
MPI_sendrecv	Send a message and wait for reply
MPI_issend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there are none
MPI_irecv	Check if there is an incoming message, but do not block

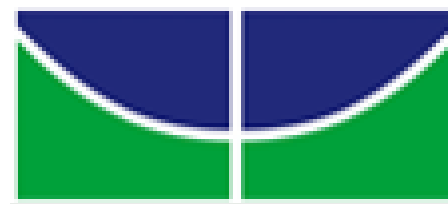


5. Programação em SHM

Sistemas de memória compartilhada



- Thread Posix
- OpenMP (primitivas de paralelização do código via compilador)



PSPD

Prof. Fernando W Cruz